

A Kernel-Checked Exclusion for Erdős Problem 647 to 10^{13} via a modular sieve proved inside the kernel and a sparse witness walk

Eero Tölö
Fjordmind Labs, Oslo, Norway
eero@fjordmind.no

October 2026

Abstract

Erdős Problem 647 asks whether any $n > 24$ satisfies $\max_{m < n} (m + \tau(m)) \leq n + 2$, where τ is the divisor-count function. The largest range excluded end to end by a proof kernel was $(24, 10^9]$ (Mian–Siddique, *decanus*), via a contiguous chain of 6.7 million factorization witnesses that the authors describe as at the ceiling of the representation. We extend the kernel-checked exclusion to $(24, 10^{13}]$, a $10,000\times$ larger range, with a *smaller* certificate: 866,484 witnesses in about 21 MB of Lean source. The gain comes from proving the structural reduction inside the kernel first: four modular sieve stages, proved in Lean 4 and checked by its kernel with no appeal to compiled-code evaluation, force $360360 \mid n$ for every candidate $n > 10^5$ and confine $(n \bmod 17, 19, 23, 29)$ to 4,431 of 215,441 residue classes for $n > 10^{11}$. Only the surviving grid points then need a witness, and each needs one. The resulting theorem is stated in the vocabulary of the public formal statement and depends on nothing beyond the three standard axioms of Lean’s mathematical library. No new mathematics is claimed: the sieve stages re-derive, fully kernel-checked, a residue reduction that had previously been verified only by trusting compiled code. The contribution is the certificate architecture, together with a cost model showing where this family of methods stops: a kernel-checked horizon of 10^{17} – 10^{18} , not the 10^{22} unverified GPU frontier.

1 Introduction

Let $\tau(m)$ denote the number of divisors of m . Problem 647 in Bloom’s database of Erdős problems (Erdős–Selfridge) asks whether some $n > 24$ satisfies

$$\max_{m < n} (m + \tau(m)) \leq n + 2.$$

The condition holds at $n = 24$ and the problem is open. Computational exclusions reach 10^{12} by direct sieve (Idén), about 6.2×10^{17} via Hughes’s modular reduction (Lean with compiled-code evaluation, residue classes searched on a GPU), about 9.2×10^{18} (bentrd), and 10^{22} (GPU with CRT lifting, 2026). None of these is checked by a proof kernel. Mian and Siddique [1] gave the first kernel-checked exclusion, at 10^9 , and framed the natural next step as “structure rather than length”: certify the modular reduction inside the kernel so that only surviving classes need witnesses. This note aims to do that.

Reading the Lean vocabulary. The proofs are written in the Lean 4 proof assistant on top of its mathematical library Mathlib [4]. A few conventions from that world recur below, and we fix them here so that the note can be read without Lean background.

- *Kernel.* Lean separates proof *search* (tactics, automation, user code) from proof *checking*. Every proof, however it was found, is re-checked by a small trusted core, the kernel. “Kernel-checked” means exactly this: the only thing one has to trust is the kernel.
- *Typewriter names.* A name in typewriter font, such as `sup_iff_cand` or `kill0k`, is the identifier of a definition or theorem in the Lean source. We give each one in words at first use; the name is there so that a reader can locate the exact statement in the repository, and nothing more.
- *Axioms.* Lean can print the complete list of axioms a theorem depends on (the command `#print axioms`). Ordinary classical mathematics in Mathlib rests on three: propositional extensionality (`propext`), the axiom of choice (`Classical.choice`) and quotient soundness (`Quot.sound`). “Three standard axioms” and “kernel-clean” below both mean that a theorem’s list is exactly these three and nothing else.
- *native_decide.* Lean offers a shortcut that evaluates a decidable statement by compiling it to native machine code and trusting the result. It is fast, but it adds an extra axiom (“trust the compiler”) to the theorem’s list, so the kernel is no longer the only trusted component. We never use it; wherever this note says “compiled-code trust” it means a proof that does. The honest alternative, `decide`, has the kernel itself carry out the computation, which is slower but fully checked.
- *Tactics.* `omega` is Lean’s decision procedure for linear integer arithmetic; `by_cases` is a case split on a proposition. Both produce ordinary kernel-checked proofs. `sorry` is a placeholder for a missing proof; a development with none is complete.
- *Certificates.* A *witness* or *certificate* is data (for us: a number, a shift, and a factorization) together with a once-proved *soundness lemma* saying “if this data passes a Boolean check, the mathematical statement holds”. The kernel then verifies each item by running the check with `decide`. “Witness” is used in its usual logical sense: an explicit object that establishes an existential statement, here the $m < n$ that shows n is not a solution.
- *Source vocabulary.* The Lean source, following decanus, calls an excluded number “killed” and calls each modular stage a “rung”. In prose we say *excluded* and *stage*; decanus itself uses “rung” for its range tiers (10^7 , 10^8 , 10^9), so the word would otherwise carry two meanings here. Identifiers such as `kill0k` and `Rung4` are quoted as they are in the repository.

Result.

$$\text{erdos647_no_solution_upto_1e13} : \forall n : \mathbb{N}, 24 < n \rightarrow n \leq 10^{13} \rightarrow \neg \left(\bigsqcup_{m : \text{Fin } n} (m + \sigma_0(m)) \leq n + 2 \right)$$

Here $\bigsqcup$ is Lean’s notation for a supremum over an index type, $\text{Fin } n$ is the type $\{0, \dots, n - 1\}$ and σ_0 is Mathlib’s name for the divisor-count function τ (Lean writes function application without parentheses, $\sigma \ 0 \ m$; we write $\sigma_0(m)$ and $\tau(m)$ throughout). The inner proposition

is therefore the literal $\max_{m < n} (m + \tau(m)) \leq n + 2$, and the theorem says that no n in $(24, 10^{13}]$ satisfies it. (The Lean source states the bound as the literal 10000000000000; the file `Erdos647Sparse/Headline13.lean` in the repository has the exact text.) Lean 4.30.0 / Mathlib v4.30.0; the axiom report lists exactly the three standard axioms. The inner predicate is the one pinned by decanus from `google-deepmind/formal-conjectures` (`ErdosProblems/647.lean`, commit `c252a41`), so the theorem speaks the public formal statement’s vocabulary and composes with the decanus range theorems. Source, certificates, generators and verification record: <https://github.com/fjordmindlabs/math> (directory `erdos647-sparse`).

Trust tier. As in decanus, this is a certification result, not a computational record; the range is nine orders of magnitude below the largest unverified computation. The claim is that every step from 24 to 10^{13} —the divisibility forced by each sieve stage, the residue classes removed, the divisor bound of every witness, the primality of every listed factor, and the gapless coverage of the surviving grid—is checked by the Lean kernel.

Contributions.

1. *A kernel-clean modular sieve* (§3): $2520 \mid n$ ($n > 1800$), $360360 \mid n$ ($n > 10^5$), a 41-class survivor list for $(n \bmod 17, n \bmod 19)$ ($n > 2.5 \times 10^8$), and a 4,431-class survivor list for $(n \bmod 17, 19, 23, 29)$ ($n > 10^{11}$). The 41-class list is an independent re-derivation of Hughes’s reduction mod 46189 with the compiled-code trust removed.
2. *Sieve stages as certificate lists, not tactic proofs* (§3): the 22,916 class exclusions of the fourth stage are data checked by chunked kernel computation under one generic soundness lemma, the same shape as the witness certificate.
3. *A sparse-walk certificate* (§4): one witness per surviving grid point; 866,484 witnesses over $(10^7, 10^{13}]$ versus 6,685,922 for decanus’s contiguous $(24, 10^9]$.
4. *Engineering for kernel scale* (§5): bitmask residue filters, chunk sizing from measured superlinear fuel cost, and a balanced binary-split driver after a linear case-split ladder went quadratic at 1,525 chunks.
5. *A scaling analysis* (§7): what deeper sieve stages buy, a structural argument that no prime ≥ 17 is ever fully forced by this method, and a cost curve to 10^{22} ; and an outline, not a result, of how the same method would reach about 10^{18} (§8).

2 The divisor-budget engine

For $n \in \mathbb{N}$ write $\text{Cand}(n)$, “ n is a candidate”, for the proposition $\forall m < n, m + \tau(m) \leq n + 2$: n satisfies the condition of Problem 647. The problem asks whether $\text{Cand}(n)$ holds for some $n > 24$, and excluding a range means proving $\neg \text{Cand}(n)$ on it. A bridge lemma (`sup_iff_cand` in the source) shows that $\text{Cand}(n)$ is equivalent to the supremum form used in the public statement, $\bigsqcup_{m: \text{Fin } n} (m + \sigma_0(m)) \leq n + 2$; both directions are routine facts about the supremum of a finite range, with $n = 0$ handled separately because the range is then empty. All internal work is on Cand .

The single engine behind every stage is the *divisor budget*: $\text{Cand}(n)$ gives, for every shift $1 \leq k \leq n$, $\tau(n - k) \leq k + 2$. If $m = d \cdot t$ with $d < t$ then $\tau(m) \geq 2\tau(d)$, since the divisors of d and their t -multiples are disjoint by size (lemma `two_mul_tau_le`). Hence:

Lemma 1 (source name `rung`). *If $\text{Cand}(n)$, $k \geq 1$, $d \mid n - k$ and $d^2 < n - k$, then $2\tau(d) \leq k + 2$.*

Contrapositively, a shift k whose forced divisor d of $n - k$ satisfies $2\tau(d) \geq k + 3$ excludes n . Every stage is a table of (residue class, shift k , forced divisor d) with $2\tau(d) \geq k + 3$; all tables were enumerated and verified in Python first and then generated as Lean.

3 The modular sieve

Stage 1: $2520 \mid n$ for $n > 1800$ (**theorem** `mod2520_of_cand`). For each nonzero residue of n modulo 8, 3, 9, 5, 7 there is a shift $k \leq 6$ at which already-established divisibilities stack into some $d \in \{4, 8, 6, 12, 9, 18, 5, 10, 15, 20, 7, 14, 21, 28, 35, 42\}$ with $2\tau(d) > k + 2$. One uniform case per residue, closed by linear arithmetic; the largest value the kernel has to compute is $\tau(42) = 8$.

Stage 2: $360360 \mid n$ ($n > 10^5$) and 41 survivor classes mod $17 \cdot 19$ ($n > 2.5 \times 10^8$). With $2520 \mid n$, every nonzero residue mod 11 and mod 13 is excluded at some shift $k \leq 24$ where $\gcd(2520, k)$ stacks with 11 or 13 (tight cases: $r \equiv 7 \pmod{11}$ at $k = 18$, $d = 198$; $r \equiv 7 \pmod{13}$ at $k = 20$, $d = 260$; $r \equiv 11 \pmod{13}$ at $k = 24$, $d = 312$). Mod 17 and mod 19 do not fully eliminate: per-prime survivors are $\{0, 11, 13, 14, 15, 16\} \pmod{17}$ and $\{0, 7, 11, 13, 14, 15, 16, 17\} \pmod{19}$ (48 pairs), of which 7 are excluded only by joint shifts where $17 \cdot 19 = 323$ enters one divisor (largest $d = 14535$ at $k = 45$). The 41 survivors equal, under $2520 \equiv 4 \pmod{17}$, $\equiv 12 \pmod{19}$, the 41 classes of $N = n/2520 \pmod{46189}$ in Hughes’s reduction, derived independently and kernel-clean. The threshold 2.5×10^8 is 14535^2 rounded.

Stage 4: 4,431 survivor classes mod $17 \cdot 19 \cdot 23 \cdot 29$ ($n > 10^{11}$). (The source numbers the bridge and sparse-walk step as “rung 3”, so this stage is `Rung4` in the repository.) Adding 23 and 29 jointly thins the $41 \times 23 \times 29 = 27,347$ classes to 4,431 (a $6.17\times$ sparsification; shifts $k \leq 90$; the largest forced stack is $17 \cdot 19 \cdot 23 \cdot 29 = 215,441$, whence the 10^{11} validity threshold via $d^2 < n$). The decisive design change from stage 2: *the exclusions are certificates, not tactic cases*. A certificate (i, k, ps) stores, for flat class index i , a shift k and the full factorization ps of the forced stack $d_{i,k}$; the lower bound on $\tau(d)$ is read off the factorization by decanus’s witness checker and its soundness lemma. The soundness lemma for the exclusion discharges the divisibility by the Chinese remainder theorem (one congruence per prime, combined over coprime moduli). A positional walker consumes the 27,347-class flat grid: each class is either the next exclusion certificate or the next listed survivor, and the walker’s soundness lemma yields “every class is excluded or in the survivor list”. Twenty-eight chunks of 1,000 classes, one kernel computation each (≈ 5 s). A stage-2-style tactic ladder over 27k cases would have been infeasible; the certificate form built in about five minutes.

Mod 23 alone and mod 29 alone give only $2.67\times$ and $2.16\times$; the joint stage is worth building and, per §7, is the last one that clearly is.

4 The sparse-walk certificate

After the sieve, a candidate $n \in (10^7, 10^{13}]$ is a multiple of 360360, say $n = 360360j$ with $j \in [28, 27,750,027]$, and above the respective thresholds lies in a surviving residue class. A Boolean *grid filter* encodes which j still need a witness:

- $(10^7, 10^{12}]$: the point survives if $n \leq 2.5 \times 10^8$ or n lies in one of the 41 stage-2 classes (`gridFilter` in the source); below the stage-2 threshold every multiple of 360360 needs a witness (932 extra points), above it only the 41 classes.
- $(10^{12}, 10^{13}]$: the point survives if $n \leq 10^{11}$ or both $n \bmod 323$ is a stage-2 survivor and the flat stage-4 index of n is a stage-4 survivor (`gridFilter13`). The 41-pair and 4,431-index survivor lists are stored as two natural-number bitmask literals and membership is a single bit test (Lean’s big-number arithmetic makes this fast) instead of a list scan. Bridge lemmas tie the masks to the proved stage theorems, so the headline derives “the filter accepts n ” for any candidate.

Witnesses. A surviving grid point n is excluded by one $m = n - k$ with certified $\tau(m) \geq k + 3$, in decanus’s witness format (maximal prime powers $p < 1024$, so primality of listed factors is eleven trial divisions). Because n is a multiple of 360360 its neighbourhood is divisor-rich and witnesses are cheap to find: over $(10^9, 10^{11}]$ the mean shift is $k = 1.9$, the median 1 (52.8% of points are excluded at $k = 1$, i.e. $\tau(n - 1) \geq 4$), the maximum 16; over $(10^{12}, 10^{13}]$ the mean is ≈ 1.75 and the maximum 8 on a sample. Our exclusion check (`kill0k` in the source) adds to decanus’s witness check the explicit conjunct $0 < k$: the witness check alone admits shift 0, which a countermodel found by the linear-arithmetic tactic caught during the soundness proof.

The walker. A fuel-bounded Boolean function (`sparse0k`) steps j through a chunk’s grid range; at each j it evaluates the filter and, if the point survives, requires the next witness in the list to exclude exactly $360360 \cdot j$, consuming it; non-surviving points consume nothing; leftovers are rejected. Its soundness lemma says a passing chunk excludes every surviving grid point of its range. Each chunk is a single kernel computation.

Counts. $(10^7, 10^{12}]$: 352,823 witnesses in 678 chunks of 4,096 grid indices (8.3 MB). $(10^{12}, 10^{13}]$: 513,661 witnesses in 1,525 chunks of 16,384 indices (12.3 MB). Stage-4 exclusion certificates: 22,916 (0.76 MB). Total ≈ 21 MB of generated Lean for the whole 10^{13} result, against 38.5 MB for decanus’s in-repository 10^8 chain and ≈ 260 MB for its 10^9 release asset.

Composition. $n \leq 10^7$ by decanus’s contiguous chain theorem (`killed_upto`, the in-repository 31-chunk chain); $10^7 < n \leq 10^{12}$ by the 10^{12} grid theorem; $10^{12} < n \leq 10^{13}$ by the fast-filter grid theorem together with the stage-4 and stage-2 theorems; each case is then carried back to the supremum form of the public statement by decanus’s bridge lemma (`not_sup_le_of_killed`).

5 Engineering at kernel scale

- **Chunk size from measured fuel cost.** Walk cost per grid point under the fast filter is ≈ 2.5 ms isolated and ≈ 7 ms under 8-way load; the cost of the fuel-bounded recursion is superlinear in chunk length ($\sim \text{fuel}^{1.7}$: $16,384 \rightarrow 43$ s, $32,768 \rightarrow 139$ s). Chunks of 16,384 keep the median at ≈ 90 s and the 10^{13} walk at 48.8 CPU-hours.
- **Drivers must be trees.** The theorem that glues the chunks together must split the range into cases. A 678-branch linear case-split ladder for 10^{12} elaborated in 286 s. The 1,525-branch ladder for 10^{13} reached 23 GB RSS at 21 minutes and was stopped: each negated

branch hypothesis stays in the arithmetic tactic’s context, so the ladder is quadratic. The replacement emits one span lemma per chunk and an 11-level balanced binary merge tree whose nodes each do one two-way case split with $O(1)$ context: 3,050 small theorems, 56 s.

- **Survivor membership in $O(1)$.** Deciding membership of a flat index in a 4,431-element list inside the kernel via Boolean sublist checks is quadratic and exceeded a ten-minute budget; a per-survivor proof term that points directly at the element’s position in the list is $O(1)$.
- **Modulus blow-up in linear arithmetic.** The cost of the `omega` tactic grows with the lcm of all modular facts in context; $(n-45) = 14535 \cdot ((n-45)/14535)$ proves in ≈ 10 s from exactly $\{n \bmod 45 = 0, n \bmod 17 = 11, n \bmod 19 = 7\}$ and times out if $n \bmod 2520 = 0$ is also present. Derive per-case minimal facts and clear the rest.
- **Generate, do not hand-write.** Every table is enumerated and verified in Python, then emitted as Lean; the Python verifier mirrors the semantics of decanus’s witness check, and a coverage verifier checks exact one-witness-per-surviving-point alignment over the whole grid before any Lean is emitted.

6 Verification record

Check	10^{11}	10^{12}	10^{13}
Python coverage verifier (exact alignment, every witness replayed)	35,803 OK	352,823 OK	513,661 OK
Lean build (4.30.0 / Mathlib v4.30.0)	exit 0	exit 0, 9,194 jobs	exit 0, 10,753 jobs
Axiom report on headline theorem	3 standard	same	same
Unproved placeholders or compiled-code trust in source	none	none	none
Kernel time	68×35 s	678 chunks, median 80 s	1,525 chunks, 48.8 CPU-h

A two-layer axiom gate mirroring decanus’s (a curated axiom-report manifest of 19 theorems plus a whole-library audit that recomputes the axiom list of every theorem in every `Erdos647Sparse` module from the compiled environment) passes: 8,090 theorems across 2,242 modules, all within the three standard axioms. Not yet done: a replay of the compiled modules through `lean4checker`, an independent external implementation of the kernel check, and a second from-source build on independent hardware; every build so far ran once on one 8-core desktop, the trust status decanus assigns its own 10^9 range tier.

7 What this does and does not show

Mathematical novelty: none claimed. The modular ladder (Dutta–Alexeev), the shift condition (Kitamura) and the residue reduction (Hughes) are prior work; stage 2 is Hughes’s reduction with the compiled-code trust removed, and stage 4 is its predictable extension. The problem is open and nothing here bears on whether a solution exists.

Why the small solutions are small. The complete solution set below 10^{13} is $\{2, 3, 4, 5, 6, 8, 10, 12, 24\}$. Two observations explain it and locate the difficulty. First, for a divisor k of n we have $n - k = k(n/k - 1)$, so $\tau(n - k) = 2\tau(k)$ exactly when $n/k - 1$ is prime, and $2\tau(k) \leq k + 2$ for every $k \geq 1$. The numbers 2, 4, 6, 8, 12, 24 are precisely the n for which $n/d - 1$ is prime or 1

for every proper divisor d (for 24: 23, 11, 7, 5, 3, 2, 1); we checked to 10^5 and 24 is the last such n . These are the pure form of the one residue class the budget engine can never kill (item 2 below): n divisible by every small k , with the first cofactors prime. By the Hardy–Littlewood prime-tuples heuristic that head pattern recurs infinitely often, so the first few shifts can never supply a proof. Second, a violation at shift k needs an $m = n - k$ with $\tau(m) \geq k + 3$; below 24 no integer has $\tau \geq 7$, so only shifts $k \leq 3$ can ever bite, and the small solutions are simply the n that dodge those three. From 24 onward the divisor-rich integers cover one another in a chain (24 reaches 30, 28 reaches 32, 30 reaches 36, 36 reaches 43, $\dots$), and the witness walk of §4 is that chain verified to 10^{13} . A solution is a break in the chain; the small ones lie before it starts. The cousin problem “every totative of n is prime” has the almost identical solution set $\{2, 3, 4, 6, 8, 12, 18, 24, 30\}$ and is provably finite by Bonse’s inequality, because the primorial outgrows n by a power. Here the analogous margin is only logarithmic (about $3 \log n$ expected violating shifts per surviving n), which is why no elementary growth argument is in sight.

Methodological point. Certificate architecture, not compute, moved the kernel-checked frontier four orders of magnitude at lower artifact size. The two levers were (a) proving the structure inside the kernel so that the certificate covers only the sparse survivor grid, and (b) representing the structure itself (the stage exclusions) as data checked by kernel computation under one generic soundness lemma, so that a stage costs minutes rather than a week of tactic engineering.

Limits, quantified. A sweep of the divisor-budget engine over sieve primes 17 to 97 and validity thresholds 10^{11} to 10^{21} (exact enumeration to 3×10^6 classes, Monte Carlo beyond; details in the repository appendix) gives:

1. Each added sieve prime removes a shrinking share of the remaining classes: 63% at 23, 56% at 31, 48% at 37, 44% at 41, 35% at 61, 22% at 97. A prime p helps only at shifts $k \equiv c_p \pmod{p}$, and useful k stay below about 170.
2. *Full divisibility by any prime $p \geq 17$ is never forced.* The class $c_{17} = 16$, all other residues 0, survives at every threshold: in it a sieve prime $q \neq 17$ matches shift k only if $q \mid k$, so the matched stack divides k , $d \leq k$, and the budget $2\tau(d) \geq k + 3$ would need $\tau(d) > d/2$, possible only for $d \in \{1, 2, 3, 4, 6\}$, none reachable with $k \equiv 16 \pmod{17}$. Hence 360360 is the final forced modulus; all further sieving is residue thinning, never grid coarsening.
3. Survivor-class counts grow 20–50 $\times$ per stage (4.4×10^3 at stage 4, 2.6×10^7 at stage 7 with primes ≤ 41); a kernel-checked literal above $\sim 10^8$ entries is outside anything built so far, so stage 7 is the practical ceiling of an enumerable sieve, with density floor $f \approx 2.5 \times 10^{-3}$.
4. The present walk visits every multiple of 360360 and costs $\propto T$ regardless of sieve depth; it walls at 10^{14} – 10^{15} . A *period walk* that steps only through surviving residues per period is the one structural lever left and buys about two decades: 10^{17} – 10^{18} at roughly one CPU-year (outlined in §8).
5. With any certifiable sieve, 10^{22} needs $\approx 7 \times 10^{13}$ witnesses, i.e. 10^3 – 10^4 CPU-years; even an uncertifiable 19-prime sieve leaves $\geq 4 \times 10^{11}$ witnesses. A kernel-checked 10^{22} therefore needs a different proof object—exclusions of whole arithmetic progressions at once, so that the witness count stops being $\propto T$ —which is a research question, not an engineering one.

8 Outline: a kernel-checked 10^{18}

Nothing beyond 10^{13} has been built or verified. This section records how the method would be taken to about 10^{18} , so that item 4 of §7 is concrete. The figures are cost-model estimates using the constants measured in §5; none of them is a result.

Period walk. Fix a sieve stage with prime set P and let $M = 360360 \cdot \prod_{p \in P} p$. The stage theorem, in the certificate form of §3, lists the S surviving residues $r_1 < \dots < r_S$ of $n/360360$ modulo $\prod_{p \in P} p$ above its validity threshold. A walker then visits, in each period q , only the S points $n = Mq + 360360 r_i$, and the stage theorem supplies the exclusion of every skipped point. Kernel cost becomes proportional to the number of survivors instead of to the range T . Stage 6 (primes ≤ 37) has $M \approx 8.9 \times 10^{13}$ and $S \approx 1.15 \times 10^6$, with 1.1×10^6 exclusion certificates (about 1.5 CPU-hours at the stage-4 rate) and a survivor literal $260\times$ the size of stage 4's; it is the realistic target. Stage 7 (primes ≤ 41 , $M \approx 3.7 \times 10^{15}$, $S \approx 2.6 \times 10^7$) is the ceiling of §7, item 3, and its survivor literal would be larger than anything kernel-checked so far.

Witnesses without witness data. At 10^{18} the certificate covers 7×10^9 to 1.3×10^{10} surviving points. Stored in the present format (about 24 bytes per witness) that is of the order of 10^2 GB of Lean source, which no toolchain will load; the witness data, not kernel time, is the first wall. The remedy is to move the search into the checker: for each surviving point the kernel itself trial-divides $n - k$ for $k = 1, 2, \dots$ by the primes below 1024 and stops as soon as the prime powers found certify $\tau(n - k) \geq k + 3$. This is the check `kill0k` already performs, with the factorization found rather than supplied, so the soundness lemma is unchanged and the per-chunk certificate shrinks to a range of period indices. Since shifts average 1.75, the search costs a few hundred big-number divisions per point, which we expect to be of the order of the present per-point walk cost; this has not been measured.

Estimates. At 0.5–2.5 ms of kernel time per surviving point:

T	stage	surviving points	kernel CPU time	chunks of 16,384
10^{17}	6	1.3×10^9	7–38 CPU-days	8×10^4
10^{17}	7	6.9×10^8	4–20 CPU-days	4×10^4
10^{18}	6	1.3×10^{10}	0.2–1.0 CPU-years	8×10^5
10^{18}	7	6.9×10^9	41 CPU-days–0.55 CPU-years	4×10^5

Parallel hardware divides wall time, not CPU time. The chunk counts are $40\text{--}500\times$ the 1,525 modules of the 10^{13} build and are themselves an untested build-system scale; a 20-level binary merge tree replaces the 11-level one of §5.

Where it stops. No table of residue-class exclusions, however deep, bears on the conjecture itself. For every modulus L the class $n \equiv 0 \pmod{L}$ survives: at shift k the forced divisor is $\gcd(L, k)$, which divides k , so $2\tau(\gcd(L, k)) \leq 2\tau(k) \leq k + 2 < k + 3$ for every $k \geq 1$ (every divisor of k other than k is at most $k/2$, so $\tau(k) \leq k/2 + 1$). The method therefore excludes finite ranges only. Beyond about 10^{18} the witness count outgrows kernel budgets (§7, item 5), and a proof for all n would need analytic input of a kind not currently available.

9 How this was produced

The Lean proofs, the Python generators and verifiers, the scaling analysis and the first draft of this note were written by an AI coding agent operated by Fjordmind Labs, working with compiler feedback, under the direction of the author, who set the goals, approved each extension of scope, reviewed the drafts and is responsible for the claims. We record the process because the result is small and the process is the part a reader may find informative.

Timeline. Day 1 (2026-10-01): survey of AI for research-level mathematics; selection of problem classes where a search-and-verify loop exists today (constructions, combinatorial bounds, formalization); a shortlist of Erdős problems ranked by explicit criteria. Problem 647 was chosen because its kernel-checked frontier (10^9) lagged its unverified computational frontier (10^{22}) by thirteen orders of magnitude and the decanus authors had named the next step. Day 2: stages 1–2 proved kernel-clean, the bridge to the public statement, the sparse checker and its soundness lemmas, the 10^{11} and 10^{12} walks, stage 4. Day 3: the 10^{13} walk, the scaling analysis of §7, the axiom gate, the repository and this note. Every design decision and every verification step is logged in the repository’s provenance file.

Cost. One 8-core desktop. 48.8 CPU-hours of kernel time for the 10^{13} walk, about 6 hours wall-clock; the smaller walks and the stage proofs are a fraction of that. No separate API spend: the agent ran inside the operator’s existing coding-assistant subscription. No GPU.

What went wrong and how it was caught. Three failure modes are worth recording. (i) The witness soundness lemma was first stated over a check that admitted the shift $k = 0$, under which “ $m < n$ ” is false; the `omega` tactic produced the countermodel while the soundness proof was being attempted, and $0 < k$ became an explicit conjunct of `kill0k`. (ii) The 1,525-branch linear case split for 10^{13} reached 23 GB of memory and was replaced by the balanced binary merge tree of §5; the 678-branch ladder for 10^{12} had worked and gave no warning that the next step would not. (iii) In a parallel formalization task the same week, the agent found that a failed typeclass synthesis inside a rewrite can be recovered by Lean with a synthetic `sorryAx` at exit code 0 and no `sorry` token anywhere in the source. This is why acceptance here is defined by the axiom report of every theorem, never by a clean build, and why the whole-library audit in the verification record exists. None of the three would have been visible from a successful build alone.

What the agent did not do. It did not find new mathematics, and its own analysis (§7, item 2 and the repository appendix) shows the method cannot settle the conjecture: for every modulus L the divisor budget leaves the class $n \equiv 0 \pmod{L}$ untouched (since $2\tau(k) < k + 3$ for every shift $k \geq 1$), so no finite table of sieve stages proves the statement. We think this is representative of what AI contributions to open problems look like at present: certified, incremental, cheap, and most useful when they say exactly where they stop.

Credit and provenance

Built on decanus [1] (Apache-2.0): witness format, the soundness lemmas for witnesses, chains and the bridge to the public statement, the 10^7 chain, and the pinned formal-conjectures state-

ment. Structural mathematics: Dutta–Alexeev, Kitamura, Hughes (erdosproblems.com forum, problem 647). Reference computations: Idén (10^{12}), Hughes (6.2×10^{17}), bentrdr (9.2×10^{18}), veljjanoski (10^{22}).

The proofs, code and first draft were produced by an AI agent under the author’s direction, as described in §9. Nothing in the trust argument depends on how the proofs were written: the Lean kernel checks them the same way.

References

- [1] I. Mian and S. Siddique, *A Kernel-Checked Exclusion Certificate for Erdős Problem 647*, arXiv:2608.17880 (2026). Code: <https://github.com/ibrahimmian36/decanus>.
- [2] T. Bloom, *Erdős Problems*, problem 647, <https://www.erdosproblems.com/647>.
- [3] Google DeepMind, *formal-conjectures*, FormalConjectures/ErdosProblems/647.lean, commit c252a41.
- [4] The Mathlib Community, *Mathlib*, v4.30.0, <https://github.com/leanprover-community/mathlib4>.